ISA

In A Little Message, I describe carrying √N, i√N, 1/N, ΣN and N! as types. I want to keep what a number means without immediately expanding every expression. Attaching a type and calculating its expanded value are different jobs. The tag still needs storage and control.

ISA is the instruction contract: what the program can ask the machine to do, how that request is written and what changes when it runs. I use the 192-bit Hunter design below. I keep the July 2026 instruction map in its own historical section because the two maps give different meanings to some of the same codes.

Number types

The type lives in cell 0 of `rtyp`. The other seven cells remain open. Here N is the magnitude or argument carried by the number record.

Code	Name	Meaning
0	POS	+N
1	NEG	-N
2	SQRT	√N
3	ISQRT	√(-N), or i√N for nonnegative N
4	INV	1/(+N)
5	NINV	1/(-N)
6	SIGMA	1+2+...+N
7	FACT	N!

This table describes the type design. It does not mean every type is fully executed by the current code. The archived Hunter core assigns only 0 and 1. Domain checks, zero denominators and the rules for opening the other types still need an implementation contract. A negative value is not made by silently treating the 24-bit magnitude as a signed machine integer.

Hunter instruction word

One instruction word has four xbits, or 12 bits:

```
[ OP | MOD | A | B ]
   3     3    3   3 bits
```

`OP` selects an operation family. `MOD` changes its meaning. In the arithmetic examples, A names the destination and first source register, and B names the second source. The digits 0..7 can name `R0..R7`. Other operand and addressing uses are not all settled.

OP	Family	Meaning and current boundary
0	LOAD	Load a value, without building a shadow. The source of a full-width literal is open
1	MOV	Move the number together with its shadow
2	ADD	ADD, SUB and CMP
3	MUL	SMUL, SDIV, SQRT, SQR, SHL and SHR. S means shadow
4	J	Select a conditional or unconditional jump through MOD. Target encoding open
5	LD	Read memory, as in LD R3,[R5]. Not in the code
6	ST	Write memory, as in ST [R5],R3. Not in the code
7	SYS	Stop, system and extension entry. Subcodes open

For the ADD and MUL families, MOD has these roles:

Bit	Role
0	Sign/inverse: SUB in the ADD family, SDIV in the normal MUL family
1	Extended operation
2	Suppress the ordinary result write, as in CMP

ADD uses MOD 0. SUB uses MOD 1. CMP uses MOD 5, combining subtraction with suppression of the ordinary arithmetic-result write. CMP still stores comparison/status information. Which cell receives it is open. J's MOD selects its own comparison, so I do not require a preceding CMP for every jump.

The proposed MUL extended pool is `000=SQRT`, `001=SQR`, `010=SHL`, `011=SHR`, `111=extension`. Its position in the instruction encoding is open: setting the extended bit leaves only two MOD bits, while this pool needs three. I cannot yet give every extended operation a complete machine word.

SHL and SHR shift one octal digit, or three bits. SHL corresponds to $\times 8$ within the retained width. SHR drops the last octal digit. A digit lost beyond the boundary is not preserved by changing `rpow`.

Three words I can read directly

All four-digit instruction words here are octal.

Word	Fields	Meaning
2035_8	OP=2, MOD=0, A=3, B=5	R3 = R3 + R5
2135_8	OP=2, MOD=1, A=3, B=5	R3 = R3 - R5
2535_8	OP=2, MOD=5, A=3, B=5	Compare through subtraction, suppress the ordinary result write, retain status

For a small positive example I start with `R3.rm=6`, `R5.rm=7`, both plain positive types and scale zero. `2035_8` reads those two values, sends them to the positional adder and writes 13 to R3. R5 stays 7. R3's old shadow must be updated or invalidated. This example has no magnitude overflow. It does not specify the still-open status-cell placement or instruction timing.

J conditions

I read the operands selected by A and B as a and b here.

MOD	Condition
0	a < b
1	a = b
2	a > b
3	Undefined
4	a divides b
5	b divides a
6	gcd(a,b)=1
7	Unconditional

The names `JMP=J.7`, `JE=J.1`, `JL=J.0` and `JG=J.2` are aliases in this draft. For positive single-row smooth values with `rc=1`, divisibility can be checked by comparing the corresponding prime exponents. When lower rows or rough are involved, that shortcut is insufficient and the general path is open. The proposed exponent path for the coprime condition is also open. The jump target has no assigned place in the four occupied fields.

Execution logic

Before the command, the input registers and program are available. The program counter identifies the instruction. Fetch makes its word available to control. Control reads OP, MOD and the operand selectors, then chooses the corresponding path.

```
program counter → instruction word → operand reads
                                          |
                              positional or shadow operation
                                          |
                                 result and status writes
                                          |
                        next instruction or selected jump target
```

For the ADD example, the logical work is one instruction fetch, two source-register reads, one addition and one destination-value write. Status and shadow maintenance are additional responsibilities. Those counts are not a claim about physical memory transactions or clock cycles.

A loop repeats only because a jump selects an earlier instruction. The loop's own counter or condition decides when that stops. A stop through SYS is intended, but its bit pattern is open. Until the target, stop and operand encodings are complete, I cannot claim an executable program is fully specified by this page alone. In an execution view, I expect the instruction selection, operand reads and committed result to appear as distinct states.

Open parts of the Hunter contract

LD and ST are not implemented in the archived code.

SYS subcodes and the J target encoding are open.

LOAD has no settled route for an eight-digit literal in a four-digit instruction.

The location of extended MUL subcodes is open.

The detailed cells for status, CMP output and scale are open.

General rough divisibility branches, terminal shadow encoding and `rc` interpretation are open.

Overflow behaviour, division exceptions, addressing forms and a complete timing table are not defined here.

I intend opcode selection to drive direct control paths without microcode. That intention still requires selection logic, storage and wiring. It is not a synthesis result or a measured hardware cost.

Previous version: July 2026 locked ISA

This is the earlier map locked on 9 July 2026, with v1.0.3 material dated 12 July. Its text was extracted again on 1 October. That later extraction date does not turn it into the September Hunter map.

```
[ op | MOD | dst | src/imm ]
  one xbit per field, four xbits total
```

OP	July meaning
0	LOAD, lazy shadow
1	MOV, shadow moves with the value
2	ADD family
3	MUL family
4	JMP
5	JEQ
6	JNE
7	SYS escape

In this earlier contract, large immediates use extension words. MOD bit 0 is sign/inverse, bit 1 is extended and bit 2 suppresses the ordinary result write. The extended pool lists SQRT and SQR, reserves `010/011` for CUBE/CBRT and uses `111` as an extension escape. The text also describes an operation-MOD pair of xbits.

I do not read July code 5 as Hunter LD, or code 6 as Hunter ST. I also keep July's reserved CUBE/CBRT entries apart from Hunter's proposed SHL/SHR entries. The earlier number-word and flag layout belongs to that version, not to the current 192-bit register.

CTCPU, Parts, Resolver, Türkçe

James Augustus, 2026. Licensed under CC BY 4.0. https://github.com/JamesAugustus/ctcpu-hunter